

### About Me

I am Saurav Sharma, currently a 4th-year undergraduate pursuing Computer Science at the Indian Institute of Information Technology Nagpur with a CGPA of 8.79/10.0 (expected graduation: June 2026). My technical toolkit includes Python, Django, JavaScript, Vue.js, and TypeScript, which I use to build full-stack web applications. I am an active open-source contributor and have been working with the eventyay codebase for the past several weeks.

Github : [OsauravO](#)

Gmail : [saurav1hasyo@gmail.com](mailto:saurav1hasyo@gmail.com)

**Project Title:** Convert eventyay-tickets VueJS Implementation to Single Page Application

**Possible Mentors:** hongquan, marcoag, norbusan, cweitat, mohit

### Why This Project?

I picked this project because I use eventyay and see a clear gap between the modern Video SPA (Vue 3 + Vue Router + Vuex) and the ticket system which still relies on full-page Django template rendering with jQuery. Every checkout step forces a full page reload, that is a solvable problem. Converting the ticket shop to an SPA will make the most user-facing part of eventyay feel instant and modern. I already understand the codebase deeply from my contributions and want to carry this through.

### Abstract

The eventyay ticket system – both the public ticket shop (presale) and the organizer control panel – currently runs on server-rendered Django templates with jQuery and Bootstrap 3. Every page transition triggers a full reload. This project will convert both into Vue 3 Single Page Applications built on top of the existing `/api/v1/` REST API. The presale shop comes first since it directly affects every ticket buyer: browsing events, adding to cart, and the entire checkout flow will become instant. Once that is solid, the control panel gets the same treatment starting with the most-used organizer pages (dashboard, orders, vouchers). The end result is a faster, more responsive experience on both sides, with a clean architecture that future contributors can extend.

| # | PR / Issue                                                                                                          | Description                                                                            | Status             |
|---|---------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------|
| 1 | #2692 - <a href="https://github.com/fossasia/eventyay/pull/2692">https://github.com/fossasia/eventyay/pull/2692</a> | Enhance approval workflow: CSV export, voucher bypass, and mass approval (fixes #2596) | Pending Review     |
| 2 | #2760 - <a href="https://github.com/fossasia/eventyay/pull/2760">https://github.com/fossasia/eventyay/pull/2760</a> | Replace jQuery with native Fetch API in polling scripts (fixes #2759)                  | Pending Review     |
| 3 | #2703 - <a href="https://github.com/fossasia/eventyay/pull/2703">https://github.com/fossasia/eventyay/pull/2703</a> | Fix: Allow identical ticket IDs across different events(fixes #575)                    | Ongoing Discussion |

## Features / Deliverables

**1. SPA Foundation :** I will set up a new Vue 3 app under ``webapp/tickets/``, built with Vite and wired up with Vue Router and Pinia. It follows the same structure the video app already uses, so nothing weird or experimental. Django will serve the SPA shell, and from there Vue takes over all the routing on the client side.

### 2. API Client:

The project already has a solid REST API at ``/api/v1/`` covering events, products, orders, cart, vouchers, and more. I will build a thin fetch-based client on top of it that handles auth (OAuth2 tokens for API calls, CSRF for same-origin fallback) and gives clean methods the Vue components can call directly.

### 3. Event Pages:

The organizer landing page and the event front page (product list, date picker, voucher input) will be rebuilt as Vue components. Calendar and week-calendar views for date-based events will also be covered. These replace the current Django-rendered templates without changing what the user sees, just how fast they see it.

### 4. Cart & Checkout:

This is the biggest piece. Right now every checkout step triggers a full page reload. I will turn the four-step flow (Add-ons, Questions, Payment, Confirm) into a single-page wizard with a progress bar. The cart lives in a Pinia store so adding, removing, and clearing items is instant. Payment provider forms will render inside the SPA just like they do today, but without the reload.

### 5. Order Management:

After checkout, users need to view their order, download tickets, cancel, modify, or change payment. All of these will become SPA views that talk to the existing order API endpoints. Voucher redemption and waiting list signup are included here too.

### 6. Control Panel SPA:

The organizer control panel (~100+ templates) is the second half of the tickets system. I will scaffold a separate Vue 3 app at ``webapp/control/`` sharing the same API client and build setup as the presale SPA. Within the GSoC period I will convert the highest-traffic organizer pages — the event dashboard, orders list, and single order detail — into SPA views. The remaining control panel pages (vouchers, products, settings, etc.) will stay on Django templates for now, but the routing and component architecture will be in place so they can be migrated incrementally after GSoC.

### 7. Auth & i18n:

Both SPAs need login. The presale SPA will use session auth (served from the same domain) with CSRF tokens, plus OAuth2 via ``django-oauth-toolkit`` for token-based flows. The control panel SPA will use the same session-based approach since organizers are already logged in. For translations, both apps will use ``vue-i18n`` pulling from the existing Django locale catalogs via the ``jsi18n`` endpoint.

### 8. Performance:

Each route will be lazy-loaded so the initial bundle stays small. I will add loading skeletons, error boundaries, and make sure the checkout works well on mobile.

### 9. Fallbacks:

For users with JS disabled, the SPA shell will include a ``<noscript>`` message pointing them to the legacy pages. The server still renders proper ``<title>`` and meta tags so search engines and link previews keep working.

## 10. Testing:

I will write Vitest unit tests for all Pinia stores and the API client, component tests for the cart and checkout steps, and Playwright end-to-end tests that run the full checkout flow against the Django dev server. Target is 80%+ coverage on the store and client layer.

## Timeline and Milestones:

### • May 1 - May 24: Community Bonding Period

#### ○ Preparation and Research:

- Study the video SPA architecture (webapp/video/) in depth as the reference implementation.
  - Audit /api/v1/ endpoints against presale views and identify/document any API gaps.
  - Create component wireframes for the event page, checkout flow, and order detail views.
  - Discuss API gaps and architectural decisions with mentors.
  - Continue contributing to the project by solving issues and getting existing PRs merged.
- 

### • May 25 - June 14: Phase I (Foundation)

#### ○ Weeks 1-3 (~90 hrs):

- Scaffold webapp/tickets/ using Vite 7 + Vue 3 + Pinia + Vue Router.
  - Build the API client layer for authentication, events, products, and cart management.
  - Implement the application shell including layout, navigation, event branding and locale switcher.
  - Add a Django view to serve index.html at presale URLs.
  - Implement event listing page and event detail page with product listing.
- 

### • June 15 - July 5: Phase II (Checkout Flow Implementation)

#### ○ Weeks 4-6 (~90 hrs):

- Implement cart component with add / remove / clear functionality using Pinia store.
- Develop checkout wizard flow: Add-ons → Questions → Payment → Confirm.
- Implement payment provider form rendering.
- Add voucher redemption and waiting list signup features.
- Implement error handling, loading states, and form validation.

### Midterm Evaluation (July 6 - July 10):

Working presale checkout flow implemented as a Single Page Application (SPA).

---

### • July 11 - Jul 26: Phase III (Order Management & Presale Polish)

#### ○ Weeks 7-8 (~70 hrs):

- Implement order detail, cancel, modify, and payment change views.
  - Add invoice generation and download components.
  - Implement organizer landing page and event calendar views.
  - Optimize performance using code splitting, lazy loading, and caching.
  - Improve responsive design and accessibility.
-

- **July 27 - August 9: Phase IV (Control Panel SPA)**

- **Weeks 9-10 (~70 hrs):**

- Scaffold webapp/control/ using shared Vite + Vue 3 + Pinia configuration.
  - Add a Django view to serve the control panel SPA shell.
  - Convert event dashboard, orders list, and order detail pages to SPA views.
  - Reuse the API client layer developed in Phase I.
  - Implement routing structure for remaining pages to support future migrations.
- 

- **August 10 - August 17: Phase V (Testing & Documentation)**

- **Weeks 11-12 (~50 hrs):**

- Write Vitest unit tests for all stores and API client (~80% coverage).
  - Implement Vitest component tests for key UI components.
  - Add Playwright end-to-end tests for presale checkout and control panel order views.
  - Implement progressive enhancement and <noscript> fallbacks.
  - Update doc/development/frontend.rst documentation.
  - Perform final code review, cleanup, and incorporate mentor feedback.
- 

**Final Evaluation (August 17 - August 24):**

Presale SPA complete, control panel SPA with core pages, tests, and documentation.

### Comprehensive Testing and Documentation

#### Testing Strategy:

- **Unit Tests(Vitest):** API client methods, Pinia store actions/getters, utility functions.  
Target: 80%+ coverage on stores and client layer.
- **Component Tests(Vitest + Vue Test Utils):** Product list rendering, cart add/remove, checkout step transitions, form validation, error states.
- **E2E Tests(Playwright):** Full presale checkout flow end-to-end, order management, voucher redemption, and control panel order list/detail. Run against Django dev server with test database.
- **Regression:** Verify Django fallback routes still work for JS-disabled users.
- **Performance:** Lighthouse CI checks for load time (<3s on 3G) and accessibility score (>90).

**Documentation:** It will be maintained in the repository's `doc/` folder following FOSSASIA's standard practices, updated alongside each feature milestone.

### Communication

#### 1.] FOSSASIA Engagement:

Channels: Actively participating in:

- Glitter Chat: Regular contributor (@osauravo:gitter.im).
- Mailing List: Subscribed ( email: [saurav1hasyo@gmail.com](mailto:saurav1hasyo@gmail.com)).
- Social Media: Following FOSSASIA Blogs and Twitter for updates.

#### 2.] Daily Updates: Submit scrum reports addressing:

- Progress since the last update.
- Planned tasks for the day.
- Blockers requiring mentorship support.

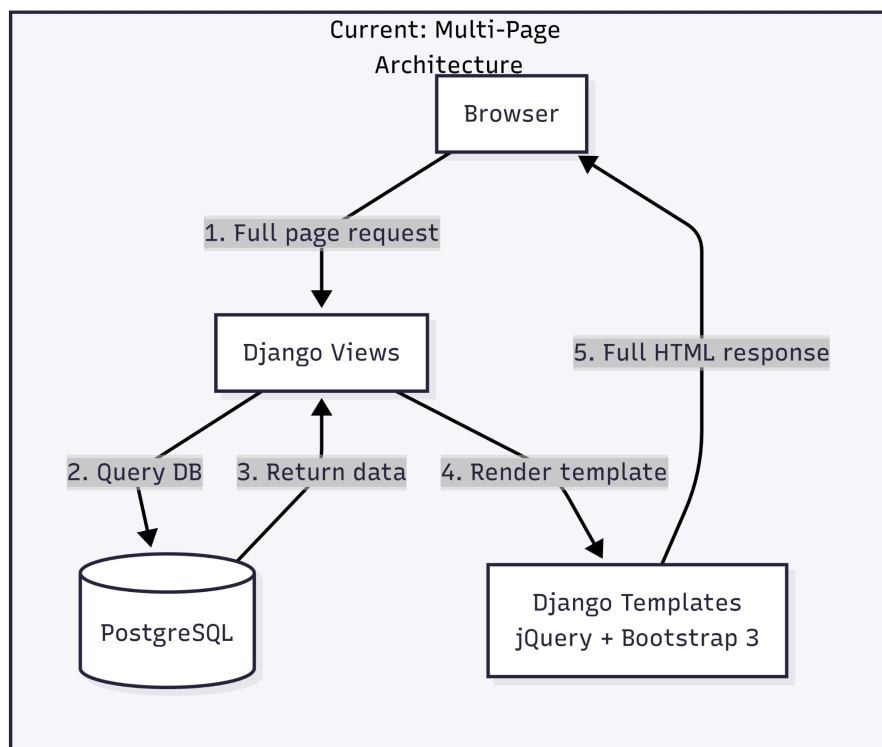
**Availability:** Fully committed to GSOC full-time (40+ hrs/week) with **no conflicting internships, travel, or obligations** during the program.

### Post GSoC Plans

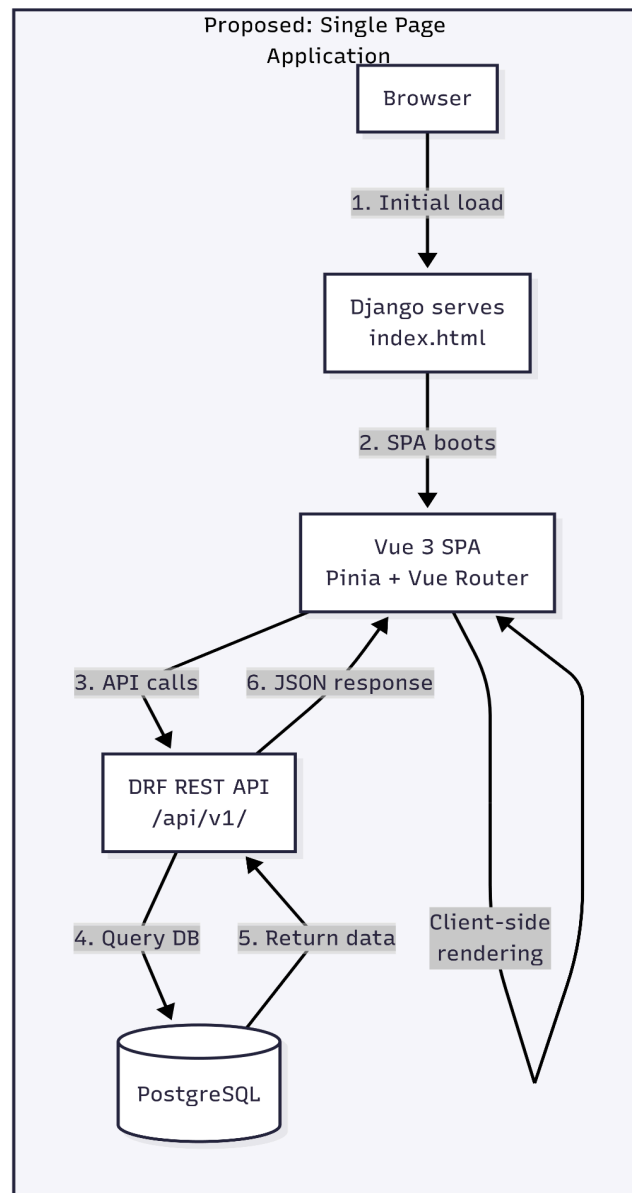
I plan to use the experience and knowledge gained from GSoC to continue contributing regularly to open source. I am fairly new to the concept. I have been building projects and sharing them publicly for over a year, but earlier I wasn't aware of the large community behind many tools I use. I want to share my work, encourage others to contribute, and gain experience by contributing to more projects. I also plan to help my juniors prepare for next year's GSoC and introduce them to open source.

### Appendix

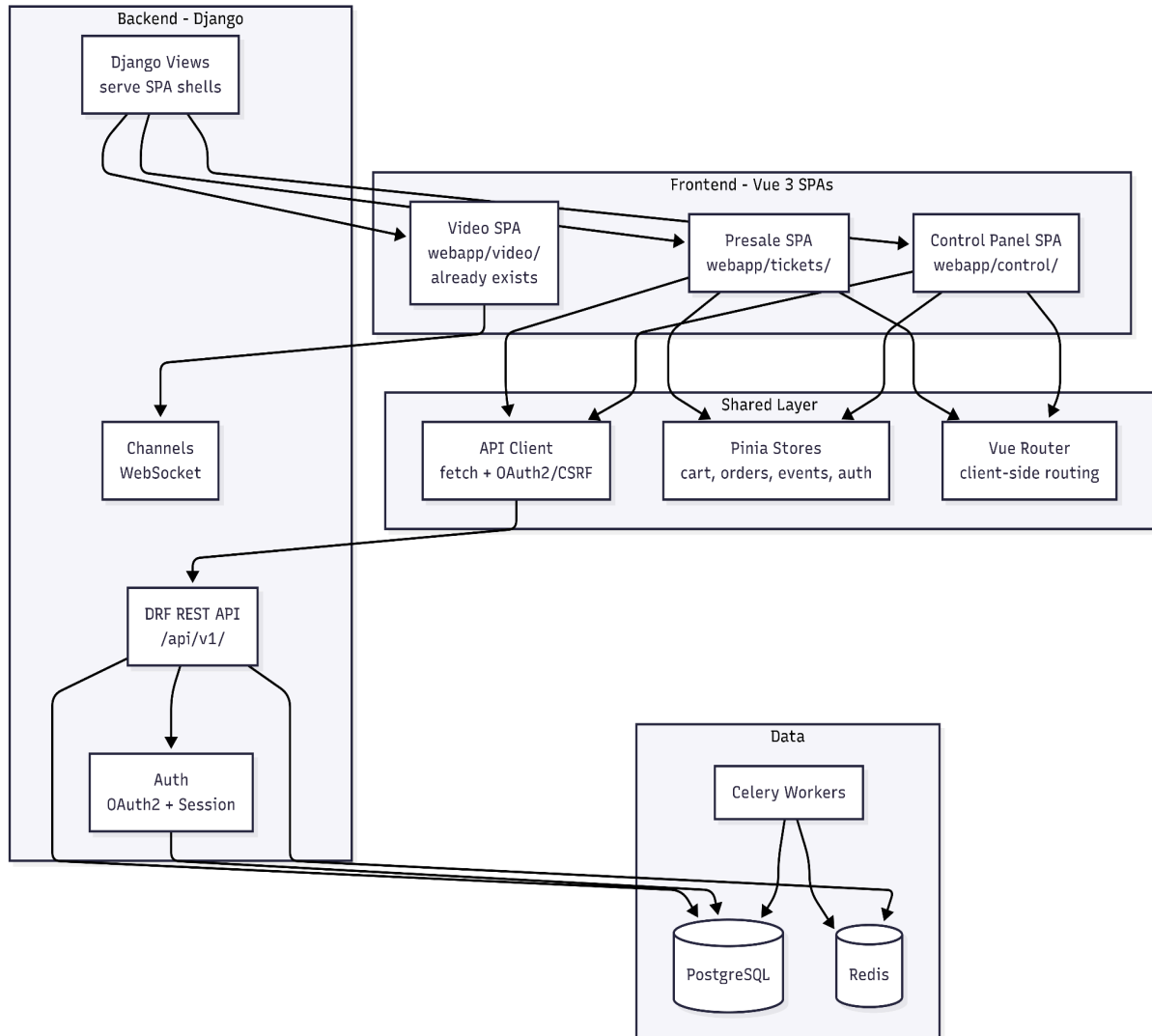
#### Current: Multi-Page Architecture



## Proposed: Single Page Application



## Full System View (both SPAs)



## Page Load Comparison

